

The OpenRouter API: A Comprehensive Technical Guide for Developers (July 2025)

Part 1: The OpenRouter API: Core Architecture and Functionality

OpenRouter has established itself as a critical piece of infrastructure in the generative AI landscape, serving as a unified gateway to a vast and ever-expanding ecosystem of large language models (LLMs). For developers, it abstracts away the complexity of a fragmented market, offering a single point of integration for accessing hundreds of models from dozens of providers. This report provides an exhaustive, granular, and up-to-date technical guide to the OpenRouter API, its full capabilities, the available models with their optimal use cases, and the parameters that can be utilized, with a specific focus on secure implementation within browser-based clients.

1.1 The Unified Interface: A Gateway to Hundreds of Models

The fundamental value proposition of OpenRouter is its function as a unified API gateway. It provides developers with access to over 400 models from more than 60 active providers, including industry leaders like OpenAI, Anthropic, Google, Mistral, and Meta, as well as a diverse range of specialized and open-source model hosts.¹ This architecture obviates the need for developers to manage a multitude of individual API keys, separate billing relationships, and disparate API request/response schemas, significantly reducing integration overhead and simplifying application logic.³

At the heart of this unified interface is a strategic adherence to the OpenAI Chat Completions API specification.⁷ This design choice makes OpenRouter a near-perfect, drop-in replacement for OpenAI, allowing developers to leverage the vast ecosystem of existing tools, SDKs, and frameworks—such as the official OpenAI Python and TypeScript SDKs, LangChain, and the Vercel AI SDK—with minimal to no code

modification.⁹ All primary API interactions are directed to the base URL

<https://openrouter.ai/api/v1>, with the POST `/chat/completions` endpoint serving as the workhorse for model inference.¹⁰

This compatibility, however, serves a purpose beyond mere convenience. It acts as a seamless onboarding ramp for the large community of developers already familiar with OpenAI's tools. Once integrated, developers are positioned within the OpenRouter ecosystem where they can access its unique, value-added features. The API request body, while accepting standard OpenAI parameters, also supports proprietary parameters like `models`, `route`, and the powerful `provider` object.⁷ These parameters unlock OpenRouter's core differentiators: intelligent routing, automated fallbacks for enhanced uptime, and sophisticated cost-performance optimization.¹⁴ Thus, the OpenAI compatibility is not just an integration feature; it is a strategic mechanism that funnels users toward the platform's more advanced capabilities.

A key component of this unified experience is schema normalization. OpenRouter processes the varied responses from different providers and standardizes them to conform to the OpenAI format. For instance, the `choices` object in a response is always returned as an array, even if the underlying model only provides a single completion.⁷ This consistency abstracts away provider-specific idiosyncrasies, ensuring a predictable and stable development experience.

1.2 Authentication, Security, and API Key Management

Securely authenticating with the OpenRouter API is a straightforward process based on industry standards. All API requests must be authenticated using a Bearer Token included in the Authorization header.¹⁰

Developers can generate and manage their API keys through the OpenRouter user dashboard.⁵ During key creation, two important configurations are available:

1. **Name/Label:** Assigning a descriptive name to each key allows for easy identification, which is particularly useful when managing keys for different applications or environments.
2. **Credit Limit:** A powerful feature for cost control, the credit limit allows you to cap the total spend for a specific API key. This is invaluable for preventing budget overruns in production or for safely distributing keys to third-party services or

end-users.¹⁰

To foster a community ecosystem and enable usage tracking, OpenRouter encourages developers to include two optional HTTP headers with their requests:

- HTTP-Referer: Your application's or site's URL.
- X-Title: A human-readable name for your application.

Passing these headers allows your application to be featured on the OpenRouter leaderboards, providing visibility and contributing to community data on model usage.¹⁰

For more advanced use cases, such as building a platform on top of OpenRouter or managing keys for a large organization, the platform provides provisioning API endpoints. The GET /api/v1/keys endpoint, for example, allows for the programmatic listing and management of your API keys.⁸

1.3 Essential API Endpoints for Application Development

While the chat/completions endpoint is the primary interface for inference, a set of utility endpoints provides the necessary metadata for building dynamic and robust applications.

- POST /chat/completions: The core endpoint for all model interactions. It accepts a JSON payload containing the model ID, a list of messages, and various other parameters to control the generation process.⁷
- GET /api/v1/models: This is an indispensable endpoint for any application that allows users to select different models. It returns a comprehensive JSON array of all available models, including critical metadata such as the model id (e.g., google/gemini-2.5-pro-preview), name, context_length, pricing details, and supported input_modalities (e.g., text, image).¹⁹ This data can be used to dynamically populate a model selection UI.
- GET /api/v1/providers: This endpoint lists all integrated model providers (e.g., OpenAI, Anthropic) and, crucially, their data privacy policies, including boolean flags for may_log_prompts and may_train_on_data.²¹ This information is vital for developers building applications with specific data privacy and compliance requirements.
- GET /api/v1/auth/key: A utility endpoint to check the status of the API key being

used for the request. The response includes the key's current usage, its limit, and information about its rate limit status.¹⁶

- GET /api/v1/generation: After a completion request is finished, the response contains a unique id. This ID can be used to query the /generation endpoint to retrieve precise, model-native token counts and cost information. This is more accurate than the normalized token counts returned in the initial chat/completions response, which are calculated using a standard tokenizer for consistency.⁷

1.4 Pricing, Credits, and Rate Limits

OpenRouter operates on a straightforward, pay-as-you-go, credit-based system with no recurring subscription fees.³ One credit is equivalent to one US dollar. Users can pre-purchase credits and add them to their account balance, with an option to set up auto-top-up to ensure uninterrupted service when the balance falls below a specified threshold.⁵

The platform's pricing model is a key consideration for developers, as are the associated rate limits, which differ significantly between free and paid tiers. This structure creates a deliberate "freemium" funnel designed to guide users from initial experimentation to production usage.

1. **Initial Exploration (Free Tier):** OpenRouter offers a wide selection of models with a :free suffix.²⁴ These models are a major draw for new users, allowing for experimentation without financial commitment. However, the rate limits are intentionally restrictive: 20 requests per minute, with a daily cap of 50 requests for new users.⁸ This is sufficient for basic testing but inadequate for any sustained application use.
2. **Initial Commitment (Increased Free Tier):** The first step in the funnel involves a small financial commitment. By purchasing at least \$10 in credits, a user's daily limit for :free models increases dramatically to 1,000 requests per day.⁸ This action both psychologically and financially invests the user in the platform while unlocking a much more usable free tier for development and low-traffic projects.
3. **Production Usage (Paid Tier):** Once an application's usage surpasses the 1,000 daily request limit, developers are naturally guided toward using the paid models. Having already integrated their application logic and billing information with OpenRouter, the transition is seamless. Rate limits for paid models are substantially higher and are primarily governed by the capacity of the underlying

provider and the user's account status.⁴

It is important to note that a low or negative credit balance can lead to 402 Payment Required errors, even when attempting to use free models.¹⁶ Furthermore, to ensure accurate billing, OpenRouter performs more frequent and aggressive database checks for accounts with low credit balances (e.g., under \$10), which can introduce additional latency to API requests. Therefore, maintaining a healthy credit balance via auto-top-up is a recommended best practice for production applications to ensure both uptime and optimal performance.²⁶

Part 2: Mastering API Parameters: From Basic Sampling to Advanced Routing

The OpenRouter API provides a rich set of parameters that allow developers to exert fine-grained control over the model's output, reliability, and cost. These parameters can be categorized into standard generation controls that mirror the OpenAI API and a powerful suite of OpenRouter-exclusive parameters for advanced routing and transformations.

2.1 Standard Generation Parameters (OpenAI Parity)

These parameters control the core sampling process of the language model and will be familiar to anyone who has worked with the OpenAI API.

- **temperature:** A floating-point number between 0.0 and 2.0 (default: 1.0) that controls the randomness of the output. Lower values (e.g., 0.2) make the output more deterministic and focused, while higher values (e.g., 1.5) increase creativity and diversity at the risk of incoherence.⁷
- **top_p:** A floating-point number between 0.0 and 1.0 (default: 1.0) that implements nucleus sampling. The model considers only the tokens comprising the top P probability mass. A value of 0.9 means that only the tokens that make up the top 90% of the probability distribution are considered for generation.⁷
- **max_tokens:** An integer that specifies the maximum number of tokens to generate in the completion. This is a hard limit to prevent overly long or runaway

responses.⁷

- **stop:** A string or an array of strings that, when encountered, will cause the model to stop generating. This is useful for forcing the model to end its output at a specific point or format.⁷
- **presence_penalty:** A floating-point number between -2.0 and 2.0 (default: 0.0). Positive values penalize new tokens based on whether they have already appeared in the response, encouraging the model to introduce new topics.⁷
- **frequency_penalty:** A floating-point number between -2.0 and 2.0 (default: 0.0). Positive values penalize new tokens based on their existing frequency in the response, making the model less likely to repeat the exact same words or phrases.⁷
- **seed:** An integer that can be used to attempt to generate deterministic, reproducible outputs. Note that reproducibility is not guaranteed across all models or providers.⁷
- **logit_bias:** A JSON object that maps token IDs to a bias value (from -100 to 100). This allows developers to manually increase or decrease the probability of specific tokens appearing in the output.⁷
- **stream:** A boolean (default: false). When set to true, the API will use Server-Sent Events (SSE) to stream back the response token by token. This is essential for creating responsive, real-time user experiences like chatbots.⁷

2.2 OpenRouter-Exclusive Parameters: Routing and Transforms

These parameters are unique to OpenRouter and provide access to its core value proposition of intelligent routing and uptime optimization.

- **models:** An array of model ID strings. This is the simplest form of fallback routing. OpenRouter will attempt to use the models in the provided order, moving to the next one in the list if the previous one fails. This is a fundamental tool for building resilient applications.⁷
- **route:** A string that defines the routing strategy. When used with the models array, a value of 'fallback' explicitly tells OpenRouter to use the list as a fallback sequence.⁷
- **transforms:** An array of strings specifying pre-defined transformations to apply to the prompt before it is sent to the model. This is an advanced feature, and developers should consult the latest OpenRouter documentation for available transforms.⁷

- **user:** A string that provides a stable, unique identifier for your end-user. This parameter is not used for billing but is crucial for helping OpenRouter's systems detect and mitigate abuse, thereby protecting the platform for all users.⁷

2.3 Advanced Provider Routing with the provider Object

For maximum control, the provider object allows for highly specific, fine-grained routing logic. This is particularly important for production applications where cost, latency, data privacy, and feature support are critical variables.⁷

- **order:** An array of provider slug strings (e.g., ["anthropic", "openai"]). This explicitly defines the priority order of providers to try for a given model request.¹⁴
- **allow_fallbacks:** A boolean (default: true). When set to false, OpenRouter will *only* attempt the providers specified in the order array. If none are available, the request will fail. This provides strict control over which infrastructure processes your data.¹⁴
- **sort:** A string that enables dynamic sorting of available providers. Valid options are "price", "latency", and "throughput". This is a powerful alternative to a fixed order, allowing the application to adapt to real-time network and market conditions.¹⁴ The :nitro suffix on a model name (e.g., mistralai/mistral-7b-instruct:nitro) is a convenient shortcut for {"sort": "throughput"}.⁸
- **quantizations:** An array of strings (e.g., ["int8", "fp16"]) that filters providers based on the quantization format of the models they serve. This is an advanced option for users with specific performance or hardware requirements.¹⁴
- **data_collection:** A string, either "allow" or "deny". This is a critical privacy control. Setting it to "deny" guarantees that the request will only be routed to providers that have a policy of not logging prompts or using them for training.¹⁴
- **require_parameters:** A boolean (default: false). If true, the request will only be sent to providers who support every parameter included in the API call. By default, providers simply ignore parameters they do not support.¹⁴

2.4 Specialized and Utility Parameters

This category includes parameters for controlling output format and enabling advanced capabilities like tool use.

- `response_format`: A JSON object used to enforce a specific output structure. The most common and widely supported value is `{ "type": "json_object" }`. When used, the model is constrained to generate a syntactically valid JSON object, which is invaluable for tasks involving structured data extraction.⁷
- `tools` & `tool_choice`: These are the core parameters for enabling function calling. They adhere to the OpenAI specification, where `tools` is an array of JSON schemas defining the available functions, and `tool_choice` can force the model to call a specific function or let it decide automatically. This functionality is discussed in detail in Part 3.5.⁷
- **Provider-Specific Parameters**: OpenRouter is capable of passing through non-standard parameters directly to the underlying provider's API. The documentation cites examples such as `safe_prompt` for Mistral models or `raw_mode` for Hyperbolic.²⁷ Because these are not standardized, developers must consult the specific model page on the OpenRouter website to see a list of supported parameters for each provider serving that model.²⁷

The following table summarizes the key OpenRouter-exclusive parameters that unlock the platform's advanced routing and management capabilities.

Table 1: OpenRouter-Specific Request Parameters

Parameter	Type	Description	Example Usage
<code>models</code>	string	A prioritized list of model IDs to try in sequence as fallbacks.	<code>{"models": ["anthropic/claude-3.7-sonnet", "openai/gpt-4o"]}</code>
<code>route</code>	string	The routing strategy to use with the models array.	<code>{"route": "fallback"}</code>
<code>transforms</code>	string	Applies pre-defined transformations to the prompt.	<code>{"transforms": ["middle-out"]}</code>
<code>user</code>	string	A unique identifier for the end-user to aid in abuse detection.	<code>{"user": "user-12345"}</code>

provider	object	An object for fine-grained control over provider selection.	<code>{"provider": {"sort": "latency", "allow_fallbacks": false}}</code>
provider.order	string	A prioritized list of provider slugs to attempt in order.	<code>{"provider": {"order": ["deepinfra", "together"]}}</code>
provider.sort	string	Dynamically sorts providers by 'price', 'latency', or 'throughput'.	<code>{"provider": {"sort": "price"}}</code>
provider.data_collection	string	Restricts routing based on provider data policies.	<code>{"provider": {"data_collection": "deny"}}</code>
provider.require_parameters	boolean	Ensures the chosen provider supports all sent parameters.	<code>{"provider": {"require_parameters": true}}</code>

Part 3: The Model Universe: A Curated Guide to Selection and Application

With a firm grasp of the API's mechanics, the next step is to navigate its most valuable asset: the vast and diverse universe of models. Choosing the right model for a specific task is paramount for building effective and cost-efficient applications. This section provides use-case-driven recommendations based on official documentation, benchmark data, and community consensus.

3.1 Navigating the Model Landscape: Providers and Categories

OpenRouter offers a staggering selection of models from a wide array of providers, ranging from the largest tech companies to specialized AI startups. Major providers include OpenAI, Anthropic, Google, Mistral AI, Meta, DeepSeek, and xAI, alongside

many others like Cohere, Together AI, and NovitaAI.⁵

To help developers navigate this landscape, the OpenRouter website provides several filtering mechanisms. Models can be sorted by their input modalities (Text, Image, File), their series or family (GPT, Claude, Gemini), their intended category (Programming, Roleplay, Marketing), and the specific API parameters they support (tools, temperature).³⁴ While the platform does not provide a single, static list, this report synthesizes information from across the platform and community to categorize key models by their optimal use cases.

3.2 Models for Deep Research, Analysis, and Synthesis

This use case involves tasks that demand sophisticated reasoning, the comprehension of complex and lengthy documents, multi-step analysis, and the ability to synthesize information from large contexts. The most suitable models for these tasks typically possess very large context windows (from 200,000 to over 1 million tokens), demonstrate top-tier performance on reasoning benchmarks like GPQA (Graduate-Level Google-Proof Q&A) and MATH, and may feature specialized "thinking" or "reasoning" modes.

The selection of a "best" model in this category is not absolute but rather a trade-off between state-of-the-art performance, cost, and latency.

- **Top-Tier Performers:** For tasks where accuracy and reasoning depth are non-negotiable, models like **Anthropic's Claude Sonnet 4** and **Opus 4**, and **Google's Gemini 2.5 Pro** represent the pinnacle of capability. They consistently lead industry benchmarks and are praised for their ability to handle complex, agentic workflows over thousands of steps.³⁵ This performance, however, comes at a premium price.
- **Balanced Options:** For many applications, models like **Anthropic's Claude 3.7 Sonnet** or **OpenAI's GPT-4.1 Mini** offer a compelling balance. They provide strong reasoning capabilities and large context windows at a significantly lower cost and latency than their top-tier counterparts, making them excellent workhorses for a wide range of analytical tasks.³⁵
- **High-Performance, Low-Cost Alternatives:** For high-volume or less mission-critical research tasks, specialized open-weight models offer remarkable capabilities at a fraction of the cost. **MiniMax's M1 (extended)** is designed for

long-context understanding and complex reasoning, while **DeepSeek's R1** and its variants are powerful, often free, alternatives that perform exceptionally well on technical benchmarks.³⁶

The following table provides a comparison of top models recommended for deep research and analysis.

Table 2: Model Recommendations for Deep Research & Analysis

Model ID	Provider	Max Context	Key Strengths & Features	Cost (Input/Output per 1M tokens)
anthropic/claude-sonnet-4	Anthropic	200,000	State-of-the-art on SWE-bench (72.7%); excels in coding and reasoning; improved autonomous navigation and instruction following. ³⁵	Varies by provider; premium pricing.
google/gemini-2.5-pro	Google	1,048,576	Top-tier performance on multiple benchmarks (LMArena #1); advanced reasoning, coding, and scientific tasks; "thinking" capabilities. ¹	~\$1.25 / ~\$10.00 (varies)
openai/gpt-4.1	OpenAI	1,048,576	Optimized for long-context reasoning, instruction following, and software engineering; high recall in	~\$2.00 / ~\$8.00

			large documents. ³⁹	
minimax/minimax-m1-extended	MiniMax	512,000	Hybrid MoE architecture; optimized for complex, multi-step reasoning, software engineering, and agentic tool use; strong benchmark performance. ³⁷	~\$0.00 / ~\$0.00 (Free)
deepseek/deepseek-r1	DeepSeek	164,000	Open-source with open reasoning tokens; performance on par with OpenAI's o1; 671B parameters (37B active). ³⁶	~\$0.00 / ~\$0.00 (Free) or paid variants

3.3 Models for Vision, Imaging, and Multimodal Tasks

It is essential to distinguish between two types of image-related tasks: image input (multimodality) and image output (generation). The OpenRouter API has native support for the former but not the latter.⁴²

Image and PDF Input: The POST /chat/completions endpoint is fully multimodal. It can accept images and PDFs by structuring the content field of a user message as a multi-part array.⁷

- **Images** can be provided either as a publicly accessible URL or as a base64-encoded string. Supported formats include PNG, JPEG, and WebP.⁴³
- **PDFs** are also sent as base64-encoded data. For models that do not natively support file inputs, OpenRouter provides its own parsing engines (mistral-ocr for scanned documents and pdf-text for text-based PDFs) which incur a per-page

processing fee.⁴³

The leading multimodal models capable of processing these inputs include:

- openai/gpt-4o and openai/gpt-4.1³⁹
- anthropic/claude-3.7-sonnet³⁶
- google/gemini-2.5-pro³⁸
- mistralai/pixtral-12b (Mistral's first dedicated text+image-to-text model)⁴⁴
- 01-ai/yi-vision⁴⁵

Image Generation: OpenRouter does not currently offer a native API endpoint for text-to-image generation. Developers seeking this functionality must use a workaround. One approach is to use a workflow automation tool like Pipedream to connect OpenRouter (for text processing) with a separate, third-party image generation API, such as Generated Photos.⁴⁶ Some community members have even built their own open-source "ImageRouter" projects to bridge this gap, demonstrating a clear demand for this feature.⁴²

3.4 Models for Web Scraping and Real-Time Information Retrieval

OpenRouter provides two primary mechanisms for grounding model responses with real-time web data, making it a powerful tool for content extraction and analysis that requires up-to-date information.

1. **Native Web Search (:online Suffix):** Several models, particularly those from Perplexity, have built-in web search capabilities. OpenRouter standardizes access to this feature via the :online model suffix (e.g., perplexity/llama-3.1-sonar-8b-online). This approach is often more powerful than using a dedicated search API because it allows the developer to leverage a superior language model (like Claude or Gemini, if they also have an :online version) for the analysis of the search results.⁴⁷
2. **Web Search Plugin:** For any model that lacks native search, OpenRouter offers a universal web search plugin powered by Exa. This plugin performs a search based on the user's prompt, retrieves relevant results, and augments the prompt with this information before sending it to the selected model. This service is billed on a per-search-result basis.⁴⁷

For more advanced or large-scale web scraping, the typical workflow involves using a

dedicated scraping tool (such as Apify, ScrapingAnt, or the open-source PAR Scrape) to extract the raw HTML or text content from target websites. This extracted content is then passed to a powerful OpenRouter model for the final step of analysis, summarization, or, most commonly, structuring the unstructured data into a clean JSON format.⁴⁹

3.5 Models for Function Calling and Agentic Workflows

Function calling (or tool use) is the mechanism that allows LLMs to interact with external systems, forming the backbone of modern agentic applications. OpenRouter provides a standardized, OpenAI-compatible interface for this capability across all supported models.³⁰ The process involves the developer defining available tools as a JSON schema, sending this schema with the request, and then handling the model's

tool_calls response by executing the specified function locally and returning the result to the model for final processing.³⁰

While the API interface is uniform, the actual performance and reliability of tool use vary significantly between models. Community feedback and testing reveal that some models, even highly capable ones, can struggle with consistently generating correctly formatted JSON for arguments or adhering to the required parameters of a tool definition.⁵⁴

The evaluation of a model's function-calling ability requires nuance. Some formal benchmarks, like the Berkeley function-calling leaderboard, are very strict, requiring perfect, single-shot JSON generation to pass.⁵⁵ In contrast, many real-world agentic workflows are built with resilience in mind, incorporating parsing logic and self-correction loops that allow a model to recover from an initial malformed attempt.³⁰ A model might score lower on a strict benchmark but perform exceptionally well in a practical agentic loop. Therefore, developers should not rely on a single benchmark score but should instead consider the benchmark's methodology alongside community anecdotes and their own empirical testing to select the most robust model for their specific agent architecture.

Table 3: Model Recommendations for Function Calling

Model ID	Provider	Key Strengths & Features	Community Feedback / Caveats	Cost (Input/Output per 1M tokens)
arcee-ai/caller-large	Arcee AI	Specialized model trained specifically for function calling; focuses on structured JSON output and multi-step tool chains. ⁵⁶	As a specialized model, it may not have the same general reasoning capabilities as larger, all-purpose models.	~\$0.55 / ~\$0.85
anthropic/claude-sonnet-4	Anthropic	State-of-the-art performance in complex, agentic workflows; reliable in following intricate instructions. ³⁵	Premium pricing. Some users have reported issues with older Claude versions on OpenRouter, but newer versions are highly praised. ⁵⁴	Varies by provider; premium pricing.
google/gemini-2.5-pro	Google	Top-tier performance on benchmarks reflecting complex problem-solving and instruction following; supports tool use natively. ³⁵	Experimental versions may be heavily rate-limited by the provider. ³⁸	~\$1.25 / ~\$10.00 (varies)
openai/gpt-4.1	OpenAI	Tuned for agent reliability and high instruction compliance (87.4% on IFEval); strong performance in IDE tooling. ⁴¹	Generally considered a reliable standard, though some users prefer Claude for coding-related tool use. ⁵⁴	~\$2.00 / ~\$8.00

mistralai/mistral-small-3.2-24b	Mistral AI	Significantly improved tool use and structured output capabilities compared to previous versions. ³⁷	As a smaller model, it may not handle extremely complex, multi-tool chains as robustly as the top-tier models.	~\$0.05 / ~\$0.10
---------------------------------	------------	---	--	-------------------

3.6 Specialized Models: Coding and Creative Writing

Beyond general-purpose tasks, OpenRouter provides access to a rich ecosystem of models that are either explicitly designed for or excel at specific domains like programming and creative writing.

Top Coding Models: Code generation, debugging, and explanation are among the most popular use cases for LLMs.

- The undisputed leader in this category, based on both benchmarks and overwhelming community sentiment, is **Anthropic's Claude series, especially Claude Sonnet 4**. It achieves a state-of-the-art 72.7% on the rigorous SWE-bench benchmark, and developers consistently report that its productivity gains are well worth the premium cost.³⁵
- **Google's Gemini 2.5 Pro and Gemini 2.0 Flash** are also top-tier contenders, frequently appearing at the top of programming and scripting leaderboards.³⁵
- For developers seeking more cost-effective solutions, the **DeepSeek family (V3 and R1)** offers exceptional performance. These models are frequently recommended in developer communities as the best free or low-cost options for coding tasks.⁵⁹

Table 4: Top-Tier Models for Code Generation

Model ID	Provider	Key Benchmark / Feature	Community Consensus	Cost (Input/Output per 1M tokens)
anthropic/claude-sonnet-4	Anthropic	72.7% on	The "gold standard" for	Varies by provider;

		SWE-bench. ³⁵	coding; developers report significant time savings that justify the cost. ⁵⁸	premium.
google/gemini-2.5-pro	Google	Top-tier performance on coding benchmarks; excels at advanced reasoning. ³⁵	Highly capable, often compared favorably to the Claude series for complex coding tasks. ⁶⁰	~\$1.25 / ~\$10.00 (varies)
openai/gpt-4.1	OpenAI	54.6% on SWE-bench Verified; tuned for precise code diffs and agent reliability. ⁴¹	A strong and reliable choice, though some developers find Claude's outputs more intuitive for coding. ⁵⁸	~\$2.00 / ~\$8.00
deepseek/deepseek-v3-0324:free	DeepSeek	Strong performance on a variety of tasks, including coding. ⁵⁹	The go-to recommendation for a high-quality free/low-cost coding assistant. ⁶⁰	\$0 / \$0

Top Creative Writing & Roleplay Models: This category values prose quality, narrative coherence, and character consistency over pure reasoning ability.

- **Specialized Fine-tunes:** The best results often come from models explicitly fine-tuned on creative writing datasets. Models like **thedrummer/anubis-70b-v1.1**, **sophosympathe/midnight-rose-70b**, and **nothingisreal/mn-celeste-12b** are crafted for storytelling and roleplaying, excelling at generating evocative prose and maintaining character voice.³⁷
- **High-End General Models:** With the right prompting, top-tier models like **Anthropic's Claude Opus** and **Mistral Large** can also produce exceptionally high-quality creative writing.⁶³
- **Community Favorites:** In communities like SillyTavern, models such as

microsoft/wizardlm-2-8x22b are popular for being powerful, unmoderated, and offering a good balance between cost and quality for roleplaying scenarios.⁶⁴

Part 4: Implementation in Practice: Browser-Based Clients and Security

While the OpenRouter API is powerful and flexible, implementing it directly within a browser-based application (e.g., a React, Vue, or Angular single-page app) introduces a critical security challenge that is often overlooked. This section details the risks and provides the definitive, industry-standard solution for secure front-end integration.

4.1 The Critical Flaw: Exposing API Keys on the Client-Side

Any API key that is included in client-side JavaScript code is, by definition, insecure and publicly exposed.⁶⁵ This is a non-negotiable principle of web security. When a front-end application is built for production, all its code and assets are bundled into static files (HTML, CSS, JavaScript) that are sent to and executed by the user's browser.

How Keys are Exposed:

- **Source Code Inspection:** A malicious actor can simply use the browser's developer tools to inspect the application's JavaScript files and find the hardcoded API key.⁶⁶
- **Network Traffic Monitoring:** Even if the key is not immediately visible in the source, it must be sent with every API request. An attacker can monitor the browser's network tab to see the outgoing request to OpenRouter and copy the API key directly from the Authorization header.⁶⁶

Using environment variables in a front-end framework (e.g., `REACT_APP_API_KEY` in Create React App or `VITE_API_KEY` in Vite) **does not solve this problem**. These tools are designed to embed the value of the environment variable directly into the final JavaScript bundle during the build process. This is useful for managing different keys for development and production, but it still results in the key being exposed on the

client side.⁶⁸

The consequence of a stolen API key is severe. An attacker can use your key to make requests on your behalf, leading to unexpected and potentially massive charges to your account, or use your credentials to abuse the service.⁶⁵

4.2 The Solution: The Backend-for-Frontend (BFF) Proxy Pattern

The only robust and secure method for using a secret API key in a front-end application is to ensure the key never reaches the client's browser. This is achieved by implementing a Backend-for-Frontend (BFF), or proxy server, that acts as an intermediary.⁶⁵

The request flow using a proxy pattern is as follows:

1. **Client-to-Proxy Request:** The browser-based application makes an API call to an endpoint on its *own* backend server (the proxy), for example, `POST /api/generate`. This request contains the user's prompt, the selected model, and other non-sensitive parameters, but it **does not** contain the OpenRouter API key.
2. **Proxy Server Logic:** The backend server receives this request from the front-end. It then securely retrieves the OpenRouter API key from a location inaccessible to the client, such as a server-side environment variable or a dedicated secrets management service (e.g., AWS Secrets Manager, Google Secret Manager, HashiCorp Vault).⁶⁵
3. **Proxy-to-OpenRouter Request:** The proxy server constructs a new request to the actual OpenRouter API endpoint (`https://openrouter.ai/api/v1/chat/completions`). It forwards the user's original data and adds the secret `Authorization: Bearer <YOUR_OPENROUTER_KEY>` header.
4. **Response Relay:** The proxy server receives the response from OpenRouter and relays it back to the client application. If the response is streamed, the proxy server streams it back to the client.

This pattern ensures that the secret API key lives and is used exclusively on the server, completely shielded from the public-facing front-end. Modern web frameworks make this pattern easy to implement using serverless functions, such as API Routes in Next.js/Vercel or Functions in Netlify.⁷²

4.3 Solving the CORS Puzzle

Developers who attempt to call the OpenRouter API directly from a browser-based application will inevitably encounter a Cross-Origin Resource Sharing (CORS) error. CORS is a fundamental browser security mechanism that prevents a web page at one domain (e.g., <https://your-app.com>) from making requests to an API at a different domain (e.g., <https://openrouter.ai>) unless the API server explicitly allows it by sending specific CORS headers.

While it may be tempting to view this as a networking problem to be circumvented, the CORS error should be interpreted as a critical architectural warning. It is a direct symptom of an insecure implementation pattern—namely, attempting to make a cross-origin request that would require exposing a secret API key to the client.

The Backend-for-Frontend proxy pattern inherently and correctly solves the CORS issue.

- The browser makes a request to its own backend (e.g., from your-app.com to your-app.com/api/generate). This is a **same-origin** request and is not subject to CORS restrictions.
- The proxy server then makes a request to the OpenRouter API. This is a **server-to-server** request. Server-to-server communication is not bound by the browser's CORS policy.

By implementing the secure proxy pattern to protect the API key, the CORS problem is resolved as a natural and beneficial side effect. Developers should therefore treat CORS errors not as an obstacle, but as a signpost guiding them toward a more secure and robust application architecture.

4.4 Client-Side Integration with Frameworks

Once a secure proxy backend is in place, integrating it with the front-end is straightforward. The key is to point all API-calling logic to the local proxy endpoint instead of the external OpenRouter URL.

- **Using the OpenAI SDK:** When initializing the client library (e.g., OpenAI in Python

or ChatOpenAI in JavaScript), configure the `baseURL` or `basePath` option to point to your proxy endpoint (e.g., `https://your-app.com/api/proxy`). The `apiKey` parameter should be omitted or set to a non-sensitive dummy value, as the real key is now managed exclusively by the proxy server.⁷⁵

- **Using the Vercel AI SDK:** The `createOpenRouter` provider should be instantiated and used on the server-side within a Next.js API Route, which functions as the proxy. The client-side hooks like `useChat` or `useCompletion` are then configured to call this local API route, not the external OpenRouter endpoint.¹¹
- **Using Native `fetch`:** For simple implementations, a standard `fetch` call can be made directly to your proxy endpoint. The request body will contain the prompt and model information, and the front-end code will handle the JSON or streamed response returned by the proxy.⁷³

Part 5: Conclusion and Strategic Recommendations

OpenRouter has evolved from a simple convenience layer into a sophisticated and indispensable tool for developers building on generative AI. Its value proposition extends far beyond being a mere gateway to multiple models; it is a comprehensive platform for optimizing performance, ensuring uptime, managing costs, and navigating the complexities of a rapidly changing LLM ecosystem.

5.1 Synthesizing the Value Proposition: Beyond a Simple Gateway

The core strengths of the OpenRouter platform can be synthesized into three key pillars:

1. **Access & Simplicity:** By providing a single, OpenAI-compatible API for hundreds of models, OpenRouter drastically lowers the barrier to entry for experimentation and production. It abstracts away the need to manage multiple provider relationships, allowing developers to focus on building application features rather than on integration plumbing.²
2. **Resilience & Performance:** Through its fallback routing (`models` array) and intelligent provider selection (`provider` object), OpenRouter enables the creation of highly available applications that can automatically route around provider

outages or performance degradation. Features like latency and throughput-based sorting give developers the tools to fine-tune the user experience.¹⁴

3. **Optimization & Control:** The platform offers powerful levers for managing costs and privacy. Per-key credit limits, dynamic routing to the cheapest provider, and strict data collection policies (data_collection: "deny") give developers and organizations the granular control needed for production environments.¹⁴

5.2 Future Outlook (As of July 2025)

The trajectory of the generative AI landscape points to several key trends, and OpenRouter is uniquely positioned to capitalize on them.

- **The Rise of Specialized Models:** The market is moving away from a "one-model-fits-all" paradigm toward a diverse ecosystem of models fine-tuned for specific tasks (e.g., arcee-ai/caller-large for function calling, morph/fast-apply for code patching, thedrummer/anubis-70b-v1.1 for creative writing). A unified router that makes it easy to access and switch between these specialized models becomes increasingly essential.³⁷
- **The Commoditization of Inference:** As more infrastructure providers (e.g., DeepInfra, Together AI, NovitaAI) host the same popular open-weight models, they will compete primarily on price and performance (latency and throughput). This dynamic makes OpenRouter's intelligent routing features, which can automatically select the best provider in real-time, an invaluable optimization layer.²⁹
- **The Primacy of Agentic Workflows:** The future of many AI applications lies in building autonomous agents that can reason, use tools, and complete multi-step tasks. The industry's focus on improving tool use, developing models with explicit "reasoning" steps, and excelling at agentic benchmarks underscores this shift. OpenRouter's robust and standardized support for these features positions it as a critical enabler for the next generation of AI applications.³⁰

5.3 Final Recommendations for Implementation

For developers and organizations building with the OpenRouter API, the following strategic recommendations will ensure a secure, efficient, and scalable implementation.

- **Prioritize Security Above All: Never** expose an OpenRouter API key in a client-side application. The use of a backend proxy is not optional; it is a mandatory security requirement for any production system. Treat CORS errors as a warning sign of an insecure architecture and use the proxy pattern as the correct solution.
- **Start Simple, Scale with Advanced Routing:** Begin by using the simple models array parameter for basic fallback and resilience. As your application's needs mature, graduate to using the full power of the provider object to meticulously optimize for cost, latency, privacy, and performance.
- **Choose the Right Tool for the Job:** Resist the temptation to default to the largest and most expensive model for every task. Use the use-case-driven guidance in this report to make informed decisions. Select smaller, faster, and more cost-effective models for simpler tasks like classification or basic summarization, and reserve the powerful, premium models for complex reasoning, coding, or function-calling workflows where their capabilities are truly required.
- **Monitor, Measure, and Iterate:** The LLM landscape is exceptionally dynamic. A model that offers the best price-to-performance ratio today may be superseded tomorrow. Regularly use the GET /generation endpoint to retrieve accurate cost and usage data. Continuously evaluate your model choices against new releases and pricing changes to ensure your application remains optimized.
- **Engage with the Community:** The official documentation provides the "what," but the developer community often provides the "why" and "how-to" for the latest models and techniques. For the most current insights on model performance, novel prompting strategies, and emerging best practices, actively engage with the OpenRouter community on platforms like Discord and Reddit.⁷⁶

Works cited

1. OpenRouter, accessed July 2, 2025, <https://openrouter.ai/>
2. OpenRouter - AI Tool Collection - Made By Agents, accessed July 2, 2025, <https://www.madebyagents.com/tools/openrouter>
3. Can anyone help me understanding how Open Router / API key works?.. - Reddit, accessed July 2, 2025, https://www.reddit.com/r/SillyTavernAI/comments/1kuyyys/can_anyone_help_me_understanding_how_open_router/
4. OpenRouter experience : r/LLMDevs - Reddit, accessed July 2, 2025,

- https://www.reddit.com/r/LLMDevs/comments/1in9g1n/openrouter_experience/
5. OpenRouter Models - TypingMind Docs, accessed July 2, 2025, <https://docs.typingmind.com/chat-models-settings/use-openrouter-models>
 6. Understanding the Difference Between OpenRouter, LiteLLM, LangChain, and MCP | by Mahesh | May, 2025 | Medium, accessed July 2, 2025, <https://medium.com/@puneripagadioriginal/understanding-the-difference-between-openrouter-litellm-langchain-and-mcp-792ec3d63388>
 7. OpenRouter API Reference | Complete API Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/overview>
 8. OpenRouter FAQ | Developer Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/faq>
 9. llms.txt - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/docs/llms.txt>
 10. API Authentication | OpenRouter OAuth and API Keys, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/authentication>
 11. Integration Frameworks | OpenRouter SDK and Library Support, accessed July 2, 2025, <https://openrouter.ai/docs/community/frameworks>
 12. OpenRouter Quickstart Guide | Developer Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/quickstart>
 13. moe-mizrak/laravel-openrouter - GitHub, accessed July 2, 2025, <https://github.com/moe-mizrak/laravel-openrouter>
 14. Intelligent Multi-Provider Request Routing | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/features/provider-routing>
 15. OpenRouter Models - TypingMind Docs, accessed July 2, 2025, <https://docs.typingmind.com/manage-and-connect-ai-models/openrouter-models>
 16. API Rate Limits | Configure Usage Limits in OpenRouter, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/limits>
 17. List API keys | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/api-keys/list-api-keys>
 18. Chat completion | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/chat-completion>
 19. List available models | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/list-available-models>
 20. Access 400+ AI Models Through One API | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/overview/models>
 21. List available providers | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/list-available-providers>
 22. Get a generation | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/get-a-generation>
 23. OpenRouter Review 2025 - Tips, Alternatives & More - Toksta, accessed July 2, 2025, <https://www.toksta.com/products/openrouter>
 24. OpenRouter - where are all the (free) models? : r/GithubCopilot - Reddit, accessed July 2, 2025, https://www.reddit.com/r/GithubCopilot/comments/1kmaqoh/openrouter_where_are_all_the_free_models/

25. Models: 'free' - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/models/?q=free>
26. Latency and Performance | Minimizing Gateway Latency | OpenRouter | Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/features/latency-and-performance>
27. API Parameters | Configure OpenRouter API Requests, accessed July 2, 2025, <https://openrouter.ai/docs/api-reference/parameters>
28. Support Specifying OpenRouter Model Provider · Issue #5787 · infiniflow/ragflow - GitHub, accessed July 2, 2025, <https://github.com/infiniflow/ragflow/issues/5787>
29. OpenRouter - GitHub Gist, accessed July 2, 2025, <https://gist.github.com/rbiswasfc/f38ea50e1fa12058645e6077101d55bb>
30. Tool & Function Calling | Use Tools with OpenRouter, accessed July 2, 2025, <https://openrouter.ai/docs/features/tool-calling>
31. R1 - API, Providers, Stats - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/deepseek/deepseek-r1>
32. Models: 'providers' - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/x->
33. Models: 'models_feedback' | OpenRouter, accessed July 2, 2025, <https://openrouter.ai/meta>
34. Models: '}' | OpenRouter, accessed July 2, 2025, <https://openrouter.ai/>
35. LLM Rankings: programming - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/rankings/programming>
36. Models: 'mistr' - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/se>
37. Models - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/models>
38. Gemini 2.5 Pro - API, Providers, Stats - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/google/gemini-2.5-pro>
39. Models: 'mistr' - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/pe>
40. NovitaAI - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/provider/novita>
41. GPT-4.1 - API, Providers, Stats - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/openai/gpt-4.1>
42. OpenRouter for images - Medium, accessed July 2, 2025, <https://medium.com/@willteam35/openrouter-for-images-94f57444a857>
43. OpenRouter Images & PDFs | Complete Documentation, accessed July 2, 2025, <https://openrouter.ai/docs/features/images-and-pdfs>
44. Pixtral 12B - API, Providers, Stats - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/mistralai/pixtral-12b>
45. 01.AI - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/01-ai>
46. Integrate the OpenRouter API with the Generated Photos API - Pipedream, accessed July 2, 2025, <https://pipedream.com/apps/openrouter/integrations/generated-photos>
47. Web Search | Add Real-time Web Data to AI Model Responses - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/docs/features/web-search>
48. Beyond Perplexity: How Open Router Web Search Is a Game Changer | by Berto Mill, accessed July 2, 2025, <https://bertomill.medium.com/beyond-perplexity-how-open-router-web-search->

[is-a-game-changer-a971737dab05](#)

49. paulrobello/par_scrape: AI assisted web scraping and data extraction - GitHub, accessed July 2, 2025, https://github.com/paulrobello/par_scrape
50. Integrate the OpenRouter API with the ScrapingAnt API - Pipedream, accessed July 2, 2025, <https://pipedream.com/apps/openrouter/integrations/scrapingant>
51. OpenRouter - Unified LLM Interface for ChatGPT, Claude, Gemini - Apify, accessed July 2, 2025, <https://apify.com/xyzzzy/open-router>
52. Automate Google Scholar Scraping & Summarization with Python & OpenAI - Scrapingdog, accessed July 2, 2025, <https://www.scrapingdog.com/blog/automating-scraping-summarizing-google-scholar-articles-using-python-openai/>
53. Universal AI GPT Scraper - Apify, accessed July 2, 2025, <https://apify.com/louisdeconinck/ai-gpt-scraper>
54. Best models for function calling - OpenAI - Reddit, accessed July 2, 2025, https://www.reddit.com/r/OpenAI/comments/1bbc4dk/best_models_for_function_calling/
55. Why do reasoning models perform worse on function calling benchmarks than non-reasoning models ? : r/LLMDevs - Reddit, accessed July 2, 2025, https://www.reddit.com/r/LLMDevs/comments/1kbj0bg/why_do_reasoning_models_perform_worse_on_function/
56. Model Comparison - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/compare/arcee-ai/caller-large>
57. Caller Large - API, Providers, Stats - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/arcee-ai/caller-large>
58. The most used model on OpenRouter, by far, is Claude. It's also quite expensive relative to most other models. Do people not care about money? Or is Claude that good that it's worth the extra cost? : r/ChatGPTCoding - Reddit, accessed July 2, 2025, https://www.reddit.com/r/ChatGPTCoding/comments/1iekf4l/the_most_used_model_on_openrouter_by_far_is/
59. LLM Rankings | OpenRouter, accessed July 2, 2025, <https://openrouter.ai/rankings/programming/scripting>
60. whats the best coding model on openrouter? : r/RooCode - Reddit, accessed July 2, 2025, https://www.reddit.com/r/RooCode/comments/1kdu7we/whats_the_best_coding_model_on_openrouter/
61. Models: ')' | OpenRouter, accessed July 2, 2025, <https://openrouter.ai/sophosympathe>
62. Mistral Nemo 12B Celeste - OpenRouter, accessed July 2, 2025, <https://openrouter.ai/nothingiisreal/mn-celeste-12b/status>
63. What's currently the best model on openrouter for constructing ..., accessed July 2, 2025, https://www.reddit.com/r/SillyTavernAI/comments/1cvdr7b/whats_currently_the_best_model_on_openrouter_for/
64. Recommendation for OpenRouter models? : r/SillyTavernAI - Reddit, accessed

July 2, 2025,

https://www.reddit.com/r/SillyTavernAI/comments/1hhd05t/recommendation_for_openrouter_models/

65. How to Store API Keys Securely - Strapi, accessed July 2, 2025,
<https://strapi.io/blog/how-to-store-API-keys-securely>
66. Best practices for working with API keys in the frontend - JavaScript Ramblings, accessed July 2, 2025,
<https://jsramblings.com/best-practices-for-working-with-api-keys-in-the-frontend/>
67. how to secure api keys in the front-end / client side Javascript? - Stack Overflow, accessed July 2, 2025,
<https://stackoverflow.com/questions/65976152/how-to-secure-api-keys-in-the-front-end-client-side-javascript>
68. How can I securely store API keys in a public frontend project and deploy the app · community · Discussion #161673 - GitHub, accessed July 2, 2025,
<https://github.com/orgs/community/discussions/161673>
69. Is there a way to store a secret API key to my backend, which the frontend can safely ask for?, accessed July 2, 2025,
<https://softwareengineering.stackexchange.com/questions/398992/is-there-a-way-to-store-a-secret-api-key-to-my-backend-which-the-frontend-can-safely-ask-for>
70. Build an API Proxy Server - Hide Your API Keys, Rate Limiting & Caching - YouTube, accessed July 2, 2025, <https://www.youtube.com/watch?v=ZGymN8aFsv4>
71. Best way to store API keys safely and easily? : r/swift - Reddit, accessed July 2, 2025,
https://www.reddit.com/r/swift/comments/1jkzt5l/best_way_to_store_api_keys_safely_and_easily/
72. Secure API Key in Front End App using Proxy Server & User Authentication - YouTube, accessed July 2, 2025, <https://www.youtube.com/watch?v=-HJeBV70zIE>
73. How to set up OpenRouter LLM observability - PostHog, accessed July 2, 2025,
<https://posthog.com/tutorials/openrouter-observability>
74. OpenRouter Auto Llm - Relevance AI, accessed July 2, 2025,
<https://relevanceai.com/llm-models/set-up-and-use-openrouter-auto-llm-for-ai-applications>
75. OpenRouter + LangChain in JavaScript | by Maxi Perez Coto - Medium, accessed July 2, 2025,
<https://medium.com/@maxiperezco/openrouter-langchain-in-javascript-408592b4c488>
76. OpenRouter - Discord, accessed July 2, 2025,
<https://discord.com/invite/openrouter>